1022. Sum of Root To Leaf Binary Numbers

Solved

Easy | Topics | Companies | Hint

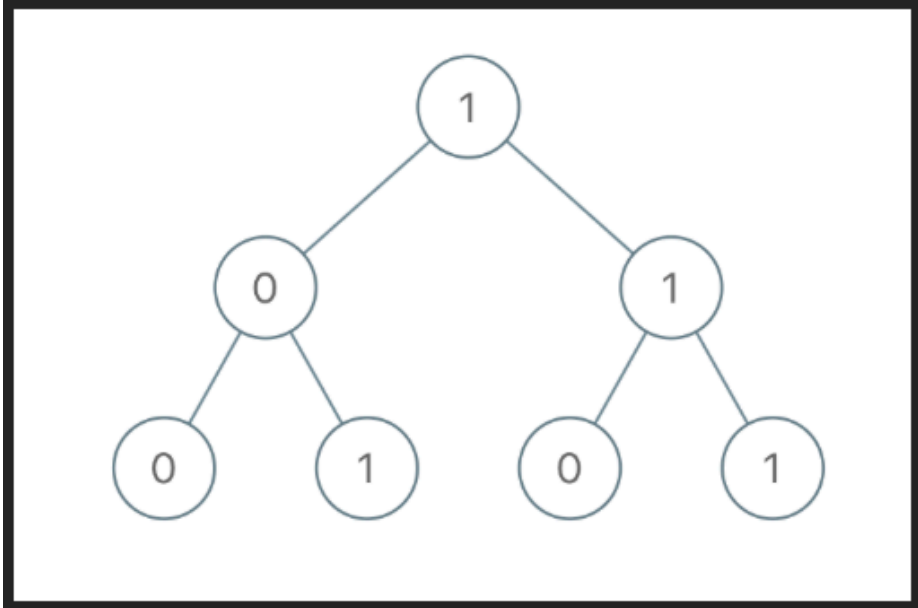
You are given the `root` of a binary tree where each node has a value `0` or `1`. Each root-to-leaf path represents a binary number starting with the most significant bit.

- For example, if the path is `0 -> 1 -> 1 -> 0 -> 1`, then this could represent `01101` in binary, which is `13`.

For all leaves in the tree, consider the numbers represented by the path from the root to that leaf. Return *the sum of these numbers*.

The test cases are generated so that the answer fits in a **32-bits** integer.

Example 1:




```
Input: root = [1,0,1,0,1,0,1]
Output: 22
Explanation: (100) + (101) + (110) + (111) = 4 + 5 + 6 + 7 =
22
```

Example 2:

```
Input: root = [0]
Output: 0
```

Constraints:

- The number of nodes in the tree is in the range `[1, 1000]`.
- `Node.val` is `0` or `1`.

Python:

```
class Solution:
    def sumRootToLeaf(self, root: TreeNode) -> int:

        def dfs(node: TreeNode, n = 0)-> None:
            if not node: return

            n = 2 * n + node.val
```

```
            if not node.left and not node.right:
                self.ans+= n
                return

            dfs(node.left , n)
            dfs(node.right, n)
            return


        self.ans = 0
        dfs(root)
        return self.ans
```

JavaScript:

```
var sumRootToLeaf = function(root) {
    function dfs(node, curr){
        if (!node) return 0;

        curr = curr * 2 + node.val;

        if (!node.left && !node.right)
            return curr;

        return dfs(node.left, curr) + dfs(node.right, curr);
    }

    return dfs(root, 0);
};
```

Java:

```
class Solution {
    public int sumRootToLeaf(TreeNode root) {
        return DFS(root, 0);
    }

    public int DFS(TreeNode rt, int x) {
        if (rt==null) return 0;
        x = x*2 + rt.val;
        if(rt.left==rt.right) return x;
        return DFS(rt.left, x) + DFS(rt.right, x);
    }
}
```